Ministerul Educaţiei și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 5

Study and Empirical Analysis of Greedy Algorithms:

Prim and Kruskal (Minimum Spanning Tree)

Elaborated:
st. gr. FAF-242 Bogdan Arama

Verified:
asist. univ. Fiștic Cristofor

Chișinău – 2026

Summary

ALGORITHM ANALYSIS

Objective

Study the greedy algorithm design technique and implement Prim's and Kruskal's algorithms for the minimum spanning tree (MST) problem. Perform empirical analysis on sparse and dense connected graphs and present the results graphically.

Tasks

1. Study the greedy algorithm design technique;
2. Implement Prim and Kruskal in a programming language;
3. Perform empirical analysis on sparse and dense graphs;
4. Increase the number of nodes and analyze the influence on performance;
5. Present the obtained data graphically and write a report.

Theoretical Notes

A **greedy algorithm** builds a solution step by step, always choosing the locally best option without revisiting past decisions. For many problems greedy choices fail; however, for MST problems, greedy strategies are correct.

Kruskal's algorithm sorts edges by weight and adds the lightest edge that does not form a cycle (Union-Find). **Prim's algorithm** grows one tree from a start vertex, repeatedly adding the cheapest edge to an unvisited vertex (priority queue).

Both algorithms are greedy and produce an MST with the same total weight on connected undirected graphs.

Introduction

Minimum spanning trees connect all vertices with minimum total edge weight. Applications include network design, clustering, and approximation algorithms. Kruskal is edge-centric; Prim is vertex-centric. Performance depends on graph density and data structures.

Comparison Metric

Primary metric: execution time $T(V)$ in seconds (`perf_counter`, minimum of 3 runs).

Secondary metric: operation count (edge relaxations / heap operations for Prim; sort and Union-Find work for Kruskal).

Both algorithms must produce the same MST weight on each graph (validation column `MST OK`).

Input Format

Connected undirected weighted graphs. Edge weights: integers 1–100.

- **Sparse:** $E = V - 1$ (random tree);
- **Dense:** $E \approx V \cdot \log_2 V$ (tree plus extra edges).

Vertex sizes: $50, 100, \ldots, 500$. Prim starts at vertex 0.

GREEDY MST ALGORITHMS

1. Kruskal's Algorithm

Overview

Kruskal sorts all edges ascending by weight and greedily adds edges that connect different components. Union-Find detects cycles efficiently.

How It Works

Initialize each vertex as its own set. For each edge in sorted order, if endpoints are in different sets, merge sets and add the edge to the MST. Stop after $V - 1$ edges.

Algorithm 1 Kruskal(G)

```
sort all edges by weight ascending
components ← Union-Find(V)
count ← 0, MST ← ∅
for each edge (u, v, w) in sorted order do
    if components.find(u) ≠ components.find(v) then
        components.union(u, v)
        add edge to MST, count ← count + 1
        if count = V − 1 then break
        end if
    end if
end for
```

Listing 1: Kruskal implementation (`algorithms/kruskal.py`)

```python
def kruskal(num_vertices, edges):
    """
    Kruskal MST on undirected weighted edges.
    edges: list of (weight, u, v)
    Returns (mst_total_weight, operations_count).
    """
    parent = list(range(num_vertices))
    rank = [0] * num_vertices
    operations = 0

    def find(x):
        nonlocal operations
        while parent[x] != x:
            operations += 1
```

```
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def union(a, b):
        nonlocal operations
        if rank[a] < rank[b]:
            parent[a] = b
        elif rank[a] > rank[b]:
            parent[b] = a
        else:
            parent[b] = a
            rank[a] += 1
        operations += 1

    sorted_edges = sorted(edges, key=lambda item: item[0])
    mst_weight = 0.0
    edges_used = 0

    for weight, u, v in sorted_edges:
        operations += 1
        root_u = find(u)
        root_v = find(v)
        if root_u != root_v:
            union(root_u, root_v)
            mst_weight += weight
            edges_used += 1
            if edges_used == num_vertices - 1:
                break

    return mst_weight, operations
```

Complexity

- **Time:** $O(E \log E)$ for sorting; Union-Find nearly $O(E\alpha(V))$.
- **Space:** $O(V + E)$.

2. Prim's Algorithm

Overview

Prim maintains a growing MST subtree. At each step it selects the minimum-weight edge connecting the subtree to a vertex outside it.

How It Works

Assign key 0 to the start vertex and ∞ to others. Repeatedly take the minimum-key vertex, mark it in the MST, and update keys of its neighbors if a lighter edge is found.

Algorithm 2 Prim(G, start)

```
key[start] ← 0, others ∞
push (0, start) into min-heap
while heap not empty do
    (w, u) ← pop minimum
    if u already in MST then continue
    end if
    mark u in MST
    for neighbor v with edge weight w_uv do
        if v not in MST and w_uv < key[v] then
            key[v] ← w_uv, push (key[v], v)
        end if
    end for
end while
```

Listing 2: Prim implementation (`algorithms/prim.py`)

```python
import heapq


def prim(adjacency_list, start=0):
    """
    Prim MST with min-heap (adjacency list of (neighbor, weight))
        .
    Returns (mst_total_weight, operations_count).
    """
    n = len(adjacency_list)
    if n == 0:
        return 0.0, 0

    start = start % n
```

```
    in_mst = [False] * n
    key = [float("inf")] * n
    key[start] = 0
    operations = 0

    heap = [(0.0, start)]

    while heap:
        weight, node = heapq.heappop(heap)
        if in_mst[node]:
            continue
        in_mst[node] = True

        for neighbor, edge_weight in adjacency_list[node]:
            operations += 1
            if not in_mst[neighbor] and edge_weight < key[
               neighbor]:
                key[neighbor] = edge_weight
                heapq.heappush(heap, (edge_weight, neighbor))

    return sum(key), operations
```

Complexity

- **Time:** $O((V + E) \log V)$ with binary heap.
- **Space:** $O(V)$.

EMPIRICAL RESULTS

Benchmarks executed with `main.py`. Table 1 shows $V = 500$. Both algorithms produced identical MST weights on every test (`MST OK = yes`).

Table 1: Benchmark results at $V = 500$.

Type	V	E	Prim (s)	Kruskal (s)	Kruskal ops
sparse	500	499	0.000226	0.000266	1281
dense	500	4482	0.001263	0.001384	5767

Execution Time Comparison

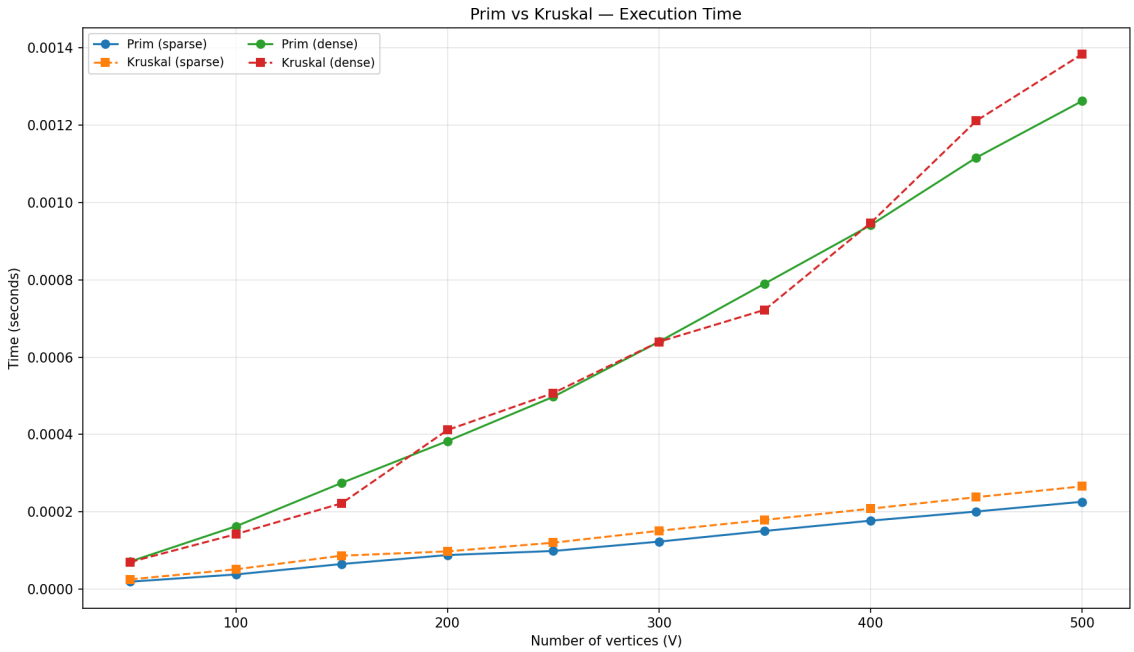

Figure 1: Prim vs Kruskal — execution time (sparse and dense).

Figure 1 shows both algorithms scale gently with V in this range. On sparse graphs ($E = V - 1$), runtimes stay below 0.0003 seconds at $V = 500$. On dense graphs, Prim and Kruskal remain close, with Kruskal sometimes slightly faster due to efficient Union-Find on sorted edges.

Execution Time by Graph Density

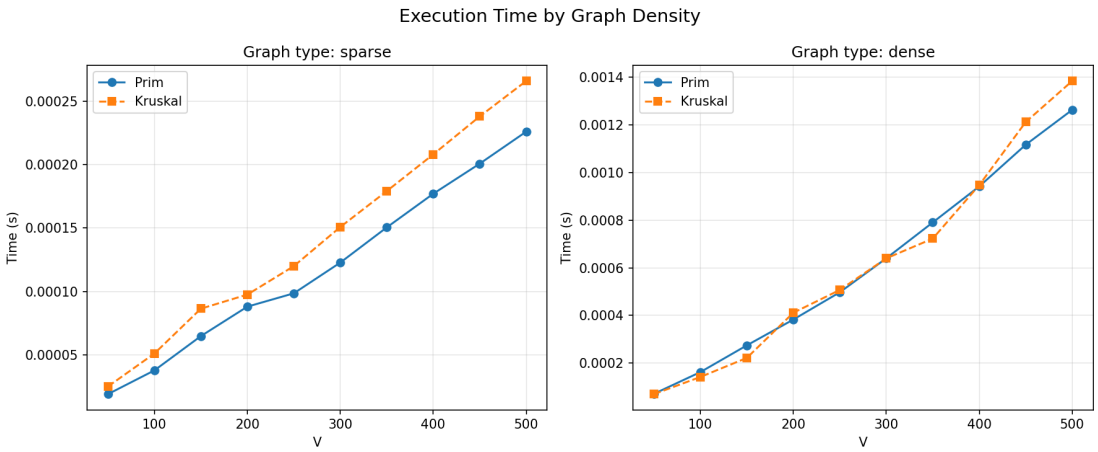

Figure 2: Execution time by graph type.

Figure 2 shows that density affects both algorithms: more edges increase sorting cost for Kruskal and heap relaxations for Prim. The gap between sparse and dense is larger than the gap between Prim and Kruskal on the same graph.

Operations Count Comparison

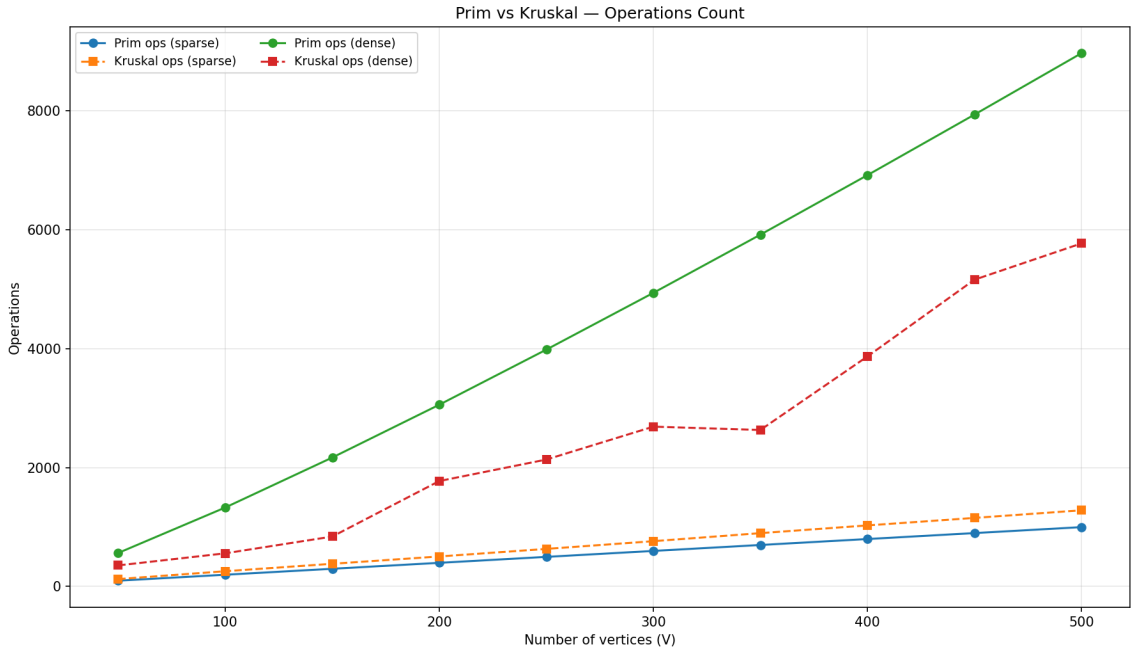

Figure 3: Prim vs Kruskal — operations count.

Figure 3 shows Prim performs more neighbor checks on dense graphs (8964 vs 5767 at $V = 500$), while Kruskal's operations include sorting overhead modeled through Union-Find iterations. On sparse trees, both remain under 1300 operations.

CONCLUSION

The empirical analysis compared Prim's and Kruskal's greedy algorithms for minimum spanning trees on connected sparse and dense weighted graphs.

Both algorithms always produced the same MST total weight, confirming correctness of the greedy approach on these inputs. Execution times remained very small for $V \leq 500$: under 0.0015 seconds even on dense graphs with thousands of edges.

On **sparse trees** ($E = V - 1$), Kruskal and Prim performed similarly, with Kruskal slightly faster in some cases because there are few edges to sort. On **dense graphs**, Prim and Kruskal remained close; Kruskal benefited from Union-Find on moderate E, while Prim paid more heap operations when degree increased.

Increasing V produced roughly linear growth in runtime for both algorithms in the tested range, consistent with $O(E \log E)$ and $O((V + E) \log V)$ behavior. Graph density had a stronger impact than the choice between Prim and Kruskal.

For sparse networks, either algorithm is suitable. For dense graphs, implementation quality and constant factors matter; Kruskal can be simpler to code, while Prim is natural when the graph is given as an adjacency list and grows from a known root.

In summary, greedy MST algorithms are fast and reliable on connected graphs; the main practical decision is data representation and whether the application needs only the MST weight or the explicit set of MST edges.

REFERENCES

1. GeeksforGeeks — *Greedy Algorithms.*
   geeksforgeeks.org/greedy-algorithms

2. TutorialsPoint — *Greedy Algorithms.*
   tutorialspoint.com/greedy-algorithms

3. Scaler — *Prim's and Kruskal's Algorithm.*
   scaler.com/topics/prims-and-kruskal-algorithm

Access the full implementation and analysis on GitHub:

https://github.com/bogdan456tech/aa-labs/tree/main/lab5